A Formalized Foundation for Research in Untyped Lambda Calculus

Alpha equivalence and de Bruijn terms in Lean 4

Chris Anto Fröschl

Munich University of Applied Sciences

Context

- Crole [2]: five textbook definitions of $=_\alpha$ define the same relation (folklore; published proof sketches are sometimes wrong).
- Norrish and Vestergaard [7]: named terms modulo $=_\alpha$ and pure de Bruijn terms are isomorphic as reduction systems (mechanized in HOL4).
- Both formalized in Lean 4 on CSLib's two untyped λ-calculi: named terms with $=_\alpha$ (#458 [9]) and de Bruijn terms with β and Church–Rosser (#475, open [8]).
- Status: Crole's Theorem 4.1 in review (#741 [4]); everything else on branches, no `sorry` except one direct proof (Thm. 4.6).
- Today: how the proofs go, in particular the parts we have not discussed yet.

Operations on atoms

- Renaming $M\langle a \mapsto b\rangle$ [LEAN]: every occurrence of a (free, bound or binding) replaced by b. Purely textual.
- Permutation action $\pi \cdot M$ [LEAN]: a bijection π on atoms applied to every atom, binders included:

  $$\pi \cdot a = \pi(a) \quad \pi \cdot (M_1\, M_2) = (\pi \cdot M_1)\,(\pi \cdot M_2) \quad \pi \cdot \lambda a.\, M = \lambda \pi(a).\, \pi \cdot M$$

  The *swap* $(a\ b) \cdot M$ [LEAN] is the case of a transposition.
- Capture-avoiding substitution $M[a := N]$ [LEAN]: replaces the free occurrences of a by N, and renames a binder to a fresh atom when it would capture a free atom of N (Curry style).

For $M = \lambda a.\, a\, b$:

$M\langle a \mapsto b\rangle = \lambda b.\, b\, b$	identifies a, b; captures the free b
$(a\ b) \cdot M = \lambda b.\, b\, a$	exchanges a and b
$M[b := a] = \lambda c.\, c\, a$	only the free b; binder renamed, c fresh

Five alpha equivalences [2, Defs. 3.1–3.5]

Swapping relations: congruence for atoms and applications, plus

$$\frac{(z\ a)\cdot M \sim (z\ b)\cdot M'}{\lambda a.\, M \sim \lambda b.\, M'}\ \mathrm{PI}$$

$\sim_{\mathrm{p}}$	$z \not\triangleright a, b, M, M'$	CSLib's $=_\alpha$
$\sim_{\mathrm{p}\#}$	$z \# a, b, M, M'$	
$\sim^{1}_{\mathrm{p}}$	$z \not\triangleright M, M'$	

Renaming relations: equivalence, congruence, plus

$$\frac{}{\lambda a.\, M \sim \lambda a'.\, M[a := a']}\ \alpha$$

$\sim_{\mathrm{r}}$	$a' \not\triangleright a, M$	Barendregt
$\sim_{\mathrm{r}\#}$	$a' \# a, M$	Hindley–Seldin

$a \not\triangleright M$: a does not occur in M at all. $a \# M$: a is not free in M.

Crole's theorem

Theorem ([2], Thms. 4.1, 4.2, 4.4–4.6)

$\sim_{\mathrm{p}} = \sim_{\mathrm{p}\#} = \sim^{1}_{\mathrm{p}} = \sim_{\mathrm{r}} = \sim_{\mathrm{r}\#}$.

4.1	$\sim_{\mathrm{p}} = \sim_{\mathrm{p}\#}$	LEAN	in CSLib PR #741
4.2	$\sim_{\mathrm{p}} = \sim^{1}_{\mathrm{p}}$	LEAN	follow-up branch
4.4	$\sim_{\mathrm{p}} = \sim_{\mathrm{r}}$	LEAN	follow-up branch
4.5	$\sim_{\mathrm{p}} = \sim_{\mathrm{r}\#}$	LEAN	follow-up branch
4.6	$\sim_{\mathrm{r}} = \sim_{\mathrm{r}\#}$		via 4.4 and 4.5

The side condition cannot be weakened much further (Prop. 4.3 LEAN):

- only $z \not\vartriangleright a, b$: $\quad \lambda a.\, a\, z \sim \lambda b.\, b\, a$
- no condition: $\quad \lambda a.\, b\, a \sim \lambda b.\, a\, b$

In both cases the free atoms differ, so the terms are not $=_{\alpha}$.

Why swap and not substitute?

- Renaming and substitution can identify two atoms: $(a\,b)[a := b] = b\,b$.
- A swap is a bijection on terms: it never identifies atoms, so it never captures, and it commutes with every operation that mentions no particular atom.
- With an unused atom swap and renaming agree: $z \not\triangleright M$ implies $(z\ a) \cdot M = M\langle a \mapsto z\rangle$ [LMN]. CSLib's $=_\alpha$ writes PI with this one-way renaming; under $z \not\triangleright M$ it is Crole's rule. $\sim_{\mathrm{p}\#}$ must keep the swap: with only $z \# M$ the two differ.

Two lemmas on permutations [2, Lemmas 6.1, 6.2]

Lemma 6.1 LEMMA

$M \sim_{\mathrm{p}} M'$ implies $(u\ v) \cdot M \sim_{\mathrm{p}} (u\ v) \cdot M'$.

- $\sim_{\mathrm{p}}$ is equivariant. Rule induction; the PI case splits on how the atoms of the rule relate to u and v: 27 cases, of which the paper shows representative ones. Lean: five helper lemmas for the representative cases and an explicit case tree covering all 27.

Lemma 6.2

Let π, π' agree on a set of atoms.

1. on $\mathrm{occ}(M)$: $\pi \cdot M = \pi' \cdot M$ LEMMA
2. on $\mathrm{free}(M)$: $\pi \cdot M \sim_{\mathrm{p}} \pi' \cdot M$ LEMMA

- Part 2 says: up to α, only the free atoms of a term matter (a support statement, see the nominal slides later). Lean: "agree on S" is Mathlib's `Set.EqOn π π' S`, for arbitrary bijections (Crole: finite support).

Proof of Theorem 4.1: $\sim_{\mathrm{p}} = \sim_{\mathrm{p}\#}$

Lean: `alphaEquiv_iff_alphaEquivPFresh`. Each relation is closed under the rules of the other; only PI needs work.

- $\sim_{\mathrm{p}} \subseteq \sim_{\mathrm{p}\#}$: $z \not\triangleright M$ implies $z \# M$, so PI is an instance of $\mathrm{PI}\#$.
- $\sim_{\mathrm{p}\#} \subseteq \sim_{\mathrm{p}}$: given $(u\ a) \cdot M \sim_{\mathrm{p}} (u\ b) \cdot M'$ with only $u \# a, b, M, M'$. Choose $z \not\triangleright u, a, b, M, M'$.
  1. Lemma 6.1 with $(z\ u)$: $(z\ u)(u\ a) \cdot M \sim_{\mathrm{p}} (z\ u)(u\ b) \cdot M'$.
  2. $(z\ u) \circ (u\ a)$ and $(z\ a)$ agree on $\mathrm{free}(M)$, since $u, z \notin \mathrm{free}(M)$. Lemma 6.2(2): $(z\ u)(u\ a) \cdot M \sim_{\mathrm{p}} (z\ a) \cdot M$.
  3. Same on the right; symmetry and transitivity give $(z\ a) \cdot M \sim_{\mathrm{p}} (z\ b) \cdot M'$, the premise of PI.
- Symmetry and transitivity of $\sim_{\mathrm{p}}$ come from #458, proved directly. Crole gets them later, from Theorem 4.4: the Lean dependency order differs from the paper's.

The renaming relations: Theorems 4.4 and 4.5

Bridge: swapping with an unused atom is, up to $\sim_{\mathrm{r}}$, substitution [2, Lemmas 6.3, 6.4]:

$$z \not\sharp M \implies (z\ a) \cdot M \sim_{\mathrm{r}} M[a := z] \quad \text{(and } \sim_{\mathrm{r}\#} \text{)}$$

- $\mathrm{PI} \Rightarrow \sim_{\mathrm{r}}$:
  $\lambda a.\, M \sim_{\mathrm{r}} \lambda z.\, M[a := z] \sim_{\mathrm{r}} \lambda z.\, M'[b := z] \sim_{\mathrm{r}} \lambda b.\, M'$ by α, BCG, α.
- $\alpha \Rightarrow \sim_{\mathrm{p}}$: for $a' \# M$ and fresh z,
  $(z\ a) \cdot M \sim_{\mathrm{p}} (z\ a') \cdot M[a := a']$ (Lemma 6.5, $n = 1$).

Departure

Crole proves the α case by induction on M, which fails for $\alpha\#$ ($M[a := a']$ may rename a binder), and repairs it with Lemma 6.5 for n swaps. We prove $n = 1$ under $a' \# M$ instead of $a' \not\sharp M$: one lemma serves both $\sim_{\mathrm{r}}$ and $\sim_{\mathrm{r}\#}$.

Open: Crole's direct proof of $\sim_{\mathrm{r}} = \sim_{\mathrm{r}\#}$ (Thm. 4.6, needs Lemma 6.7).

What the formalization had to add to [2]

- Existence of substitution (deferred in the paper): Curry-style, renaming a binder only when needed; well defined by induction on size [5, Def. 5.13].
- Lemma 6.1: all 27 cases, not representatives.
- Lemma 6.2(1): omitted in the paper.
- Lemmas 6.5 and 6.6: details omitted or "well known".
- A small change of hypotheses (Lemma 6.5, previous slide). No errors found in the paper.

The claim [7]

Two reduction systems:

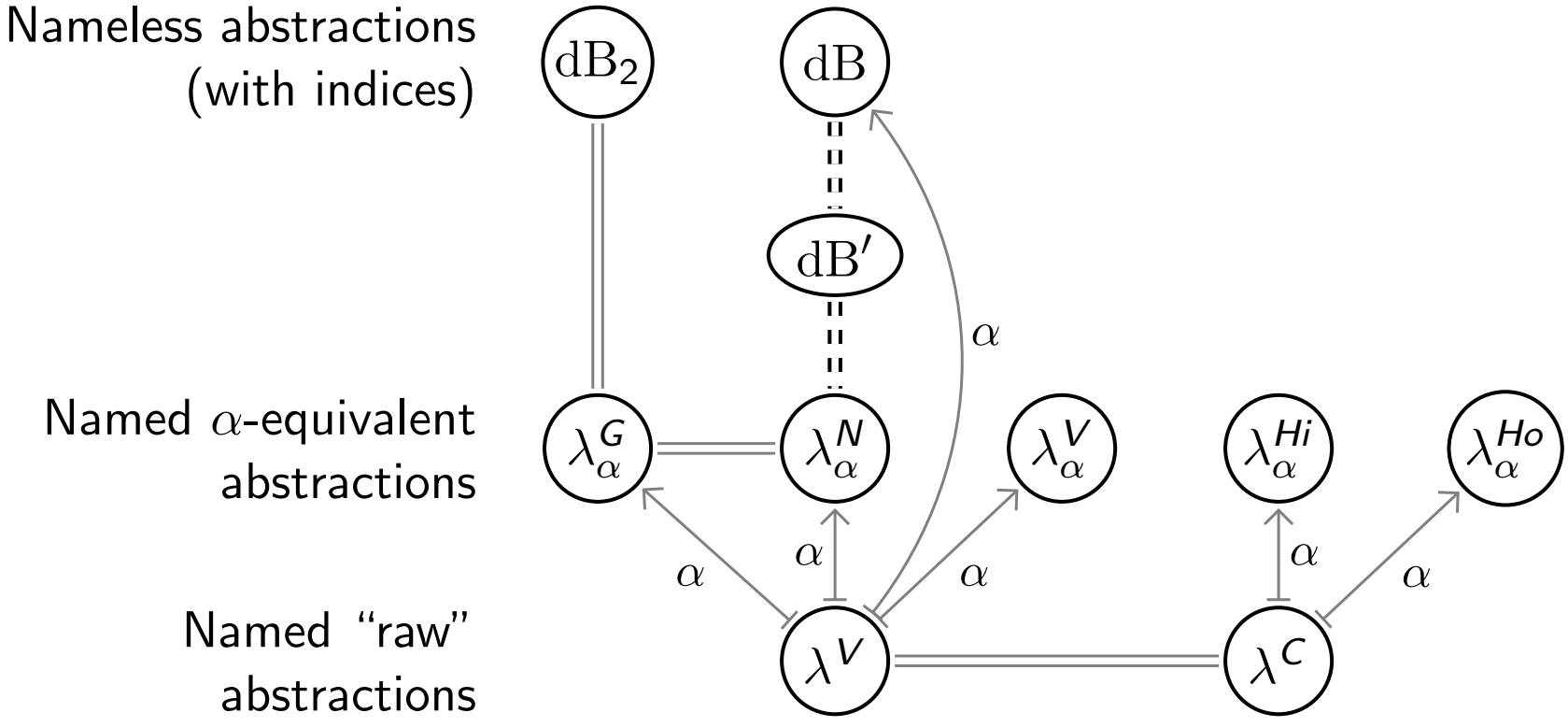
- $\langle \Lambda_\alpha, \to_\beta \rangle$: named terms modulo $=_\alpha$, ordinary β LEMN
- $\langle \mathrm{dB}, \to_\mathrm{d} \rangle$: pure de Bruijn terms, Nipkow's β LEMN

Theorem 20
There is a bijection $f : \Lambda_\alpha \to \mathrm{dB}$ with

$$M \to_\beta N \iff f(M) \to_\mathrm{d} f(N).$$

- Why "$\iff$"? For relations a bijective homomorphism is not enough: the identity $\langle \mathbb{N}, \emptyset \rangle \to \langle \mathbb{N}, < \rangle$ preserves the relation but is no isomorphism.
- Both relations: one contraction rule plus congruence rules (left, right, under a binder).

Where the result sits



Formalizations of the λ-calculus and their directly proved relationships, redrawn from [7, Figure 1]. Double lines are isomorphisms, arrows are α-quotients. The dashed lines are the results of [7], formalized here in Lean.

Reading de Bruijn terms

$$t \;::=\; \mathrm{dV}\, i \mid \mathrm{dAPP}\, t\, t \mid \mathrm{dABS}\, t$$

- A bound index counts the binders up to its own binder.
- A free index i is the i-th name of a naming context $\Gamma = [a, b, c, \ldots]$; every binder pushes its variable to the front.

Examples, with $\Gamma = [a, b, c, \ldots]$:

- $\lambda f.\, \lambda x.\, f\, x = \mathrm{dABS}\,(\mathrm{dABS}\,(\mathrm{dAPP}\,(\mathrm{dV}\,1)\,(\mathrm{dV}\,0)))$
- applied to the free $\mathrm{dV}\,5$:

$$\to_{\mathrm{d}}\;\; \mathrm{dABS}\,(\mathrm{dAPP}\,(\mathrm{dV}\,6)\,(\mathrm{dV}\,0))$$

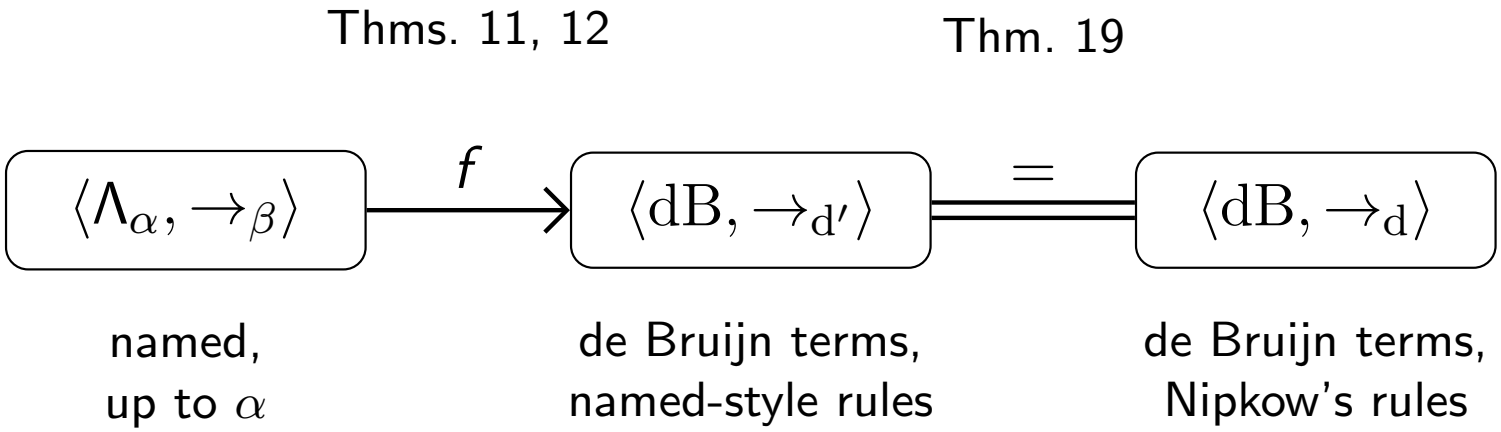
  under the remaining binder, the free 5 is now called 6.
- $(\lambda x.\, x\, a)\, b$ is $\mathrm{dAPP}\,(\mathrm{dABS}\,(\mathrm{dAPP}\,(\mathrm{dV}\,0)\,(\mathrm{dV}\,1)))\,(\mathrm{dV}\,1)$

$$\to_{\mathrm{d}}\;\; \mathrm{dAPP}\,(\mathrm{dV}\,1)\,(\mathrm{dV}\,0) \;=\; b\, a.$$

lift and nsub [6]

- lift $t\ k$ adds 1 to the free indices $\geq k$ [∃∀ℕ]: keeps them pointing at the right name under a new binder.
  - $\mathrm{lift}\,(\mathrm{dV}\,3)\,0 = \mathrm{dV}\,4, \quad \mathrm{lift}\,(\mathrm{dV}\,3)\,5 = \mathrm{dV}\,3$
  - $\mathrm{lift}\,(\mathrm{dABS}\,(\mathrm{dAPP}\,(\mathrm{dV}\,0)\,(\mathrm{dV}\,2)))\,0 =$ $\mathrm{dABS}\,(\mathrm{dAPP}\,(\mathrm{dV}\,0)\,(\mathrm{dV}\,3))$ $\quad (\lambda x.\, x\, b$: x stays, b moves)
- $\mathrm{nsub}\,s\,k\,t$ substitutes s for k and closes the gap above k [∃∀ℕ]:
  - at $k = 1$: $\mathrm{dV}\,0 \mapsto \mathrm{dV}\,0$, $\mathrm{dV}\,1 \mapsto s$, $\mathrm{dV}\,2 \mapsto \mathrm{dV}\,1$
  - $\mathrm{nsub}\,(\mathrm{dV}\,5)\,0\,(\mathrm{dAPP}\,(\mathrm{dV}\,0)\,(\mathrm{dV}\,3)) = \mathrm{dAPP}\,(\mathrm{dV}\,5)\,(\mathrm{dV}\,2)$
- Both parts are needed:
  - $(\lambda x.\, x\, a)\, b$ without the decrement gives $b\, b$.
  - $(\lambda x.\, \lambda y.\, x)\, a$: $\mathrm{nsub}\,(\mathrm{dV}\,0)\,0\,(\mathrm{dABS}\,(\mathrm{dV}\,1)) = \mathrm{dABS}\,(\mathrm{dV}\,1) = \lambda y.\, a$; without the lift inside nsub it would be $\mathrm{dABS}\,(\mathrm{dV}\,0) = \lambda y.\, y$.

The route



- dB': the same terms, but with a binder dLAM *i* over a chosen index and a substitution sub that leaves the other indices alone.
- Thm. 12: f turns $\to_\beta$ into $\to_{\mathrm{d}'}$. Easy, because dB' is built to look like Λ_α.
- Thm. 19: $\to_{\mathrm{d}'} = \to_{\mathrm{d}}$. The hard part: contraction rules by Lemma 13, binder rules by Thm. 15.

dB': a named-style binder on de Bruijn terms

- $\mathsf{sub}\, s\, k\, (\mathsf{dV}\, i) = \mathsf{if}\ i = k\ \mathsf{then}\ s\ \mathsf{else}\ \mathsf{dV}\, i$ LEMMA (no gap closing)
- $\mathsf{dLAM}\, i\, t = \mathsf{dABS}\, (\mathsf{sub}\, (\mathsf{dV}\, 0)\, (i+1)\, (\mathsf{lift}\, t\, 0))$ LEMMA (bind the free i)

dLAM forgets which name it bound, like λ on Λ_α:

- $\mathsf{dLAM}\, 0\, (\mathsf{dV}\, 0) = \mathsf{dLAM}\, 1\, (\mathsf{dV}\, 1) = \mathsf{dABS}\, (\mathsf{dV}\, 0)$ $(\lambda a.\, a,\ \lambda b.\, b)$
- $\mathsf{dLAM}\, 0\, (\mathsf{dAPP}\, (\mathsf{dV}\, 0)\, (\mathsf{dV}\, 1)) = \mathsf{dLAM}\, 2\, (\mathsf{dAPP}\, (\mathsf{dV}\, 2)\, (\mathsf{dV}\, 1)) = \mathsf{dABS}\, (\mathsf{dAPP}\, (\mathsf{dV}\, 0)\, (\mathsf{dV}\, 2))$ $(\lambda a.\, a\, b,\ \lambda c.\, c\, b)$

Every dABS term is a dLAM term over any index not free in it (Lemma 6 LEMMA).

$\to_{\mathrm{d}'}$ LEMMA: $\mathsf{dAPP}\, (\mathsf{dLAM}\, i\, t)\, u \to_{\mathrm{d}'} \mathsf{sub}\, u\, i\, t$, and congruences with dLAM i in place of dABS.

Nominal ideas, by example (1)

Slogan: only the relationships between names matter, not the names themselves.

Names a, b, c, d; permutations act on everything built from names:

- $(a\ b) \cdot a = b$, $\quad (a\ b) \cdot 5 = 5$, $\quad (a\ b) \cdot \{a, c\} = \{b, c\}$, $(a\ b) \cdot \{a, b\} = \{a, b\}$

Support: the names a permutation has to fix to fix the object.

object	support	
a	$\{a\}$	
5	$\emptyset$	
$\{a, c\}$	$\{a, c\}$	
raw term $\lambda a.\, a\, b$	$\{a, b\}$	all its names, occ
class $[\lambda a.\, a\, b]_\alpha$	$\{b\}$	free names only, free

$(a\ c)$ keeps the class $(\lambda c.\, c\, b =_\alpha \lambda a.\, a\, b)$, $(b\ c)$ does not. Quotienting by α turns "all names" into "free names" (Crole's Lemma 6.2, parts 1 and 2).

Nominal ideas, by example (2)

Equivariant: commutes with every permutation, i.e. mentions no particular name.

- Equivariant: free, size, $=_\alpha$, $\to_\beta$.
  E.g. $\lambda a.\, a =_\alpha \lambda b.\, b$, and after $(a\ c)$: $\lambda c.\, c =_\alpha \lambda b.\, b$.
- Not equivariant: "$a \in \mathrm{free}(M)$" (true for a, false for b).
- Not equivariant: $\mathrm{fresh}(S) =$ the smallest name not in S:
  $\mathrm{fresh}\{a\} = b$, but $\mathrm{fresh}\{c\} = a \neq (a\ c) \cdot b$.

Consequences for the paper:

- A function out of Λ_α may only use names equivariantly.
  $f : \Lambda_\alpha \to \mathrm{dB}$ is fine; a direct $\mathrm{dB} \to \Lambda_\alpha$ would have to invent names.
- Permutations act on de Bruijn terms through the free indices (Def. 8 LAMN); sub, dLAM and $\to_{\mathrm{d}'}$ are equivariant (Thm. 10 LAMN).

The bijection f (Thms. 11, 12)

$$f([a]_\alpha) = \mathsf{dV}\,\hat{a} \qquad f(M\,N) = \mathsf{dAPP}\,(f\,M)\,(f\,N)$$
$$f(\lambda a.\,M) = \mathsf{dLAM}\,\hat{a}\,(f\,M)$$

$\hat{a}$: the number of the atom a.

- The paper: Pitts' recursion principle for Λ_α, which needs dB as a nominal set.
- Lean: structural recursion on raw terms, shown to respect $=_\alpha$, then lifted to the quotient. Possible because dLAM forgets the name.
- Injective: dLAM is injective up to a swap. Surjective: by size, using Lemma 6.
- Substitution: $f(M[a := N]) = \mathsf{sub}\,(f\,N)\,\hat{a}\,(f\,M)$
- Thm. 12: $M \to_\beta N \iff f(M) \to_{\mathrm{d}'} f(N)$

The key observation: lift is a permutation

Lemma 14 LEAN

If all free indices of t are $\leq n$, then $\mathrm{lift}\, t\, m = \pi_{m,n} \cdot t$ for the cycle $\pi_{m,n} \colon m \mapsto m{+}1 \mapsto \cdots \mapsto n{+}1 \mapsto m$.

Example: $\mathrm{lift}\,(\mathrm{dAPP}\,(\mathrm{dV}\,0)\,(\mathrm{dV}\,2))\,0 = \mathrm{dAPP}\,(\mathrm{dV}\,1)\,(\mathrm{dV}\,3)$ is the cycle $0 \to 1 \to 2 \to 3 \to 0$; index 3 is unused in t, so it can wrap around.

Theorem 15 LEAN

For an equivariant R: R is closed under dABS $\iff$ R is closed under every dLAM i.

- dABS t and its presentation as a dLAM term differ by a permutation, and an equivariant relation cannot see permutations.
- Lemma 13 LEAN matches the contraction rules: $\mathrm{sub}\, u\, i\, t = \mathrm{nsub}\, u\, n\,(\mathrm{sub}\,(\mathrm{dV}\, n)\,(i{+}1)\,(\mathrm{lift}\, t\, n))$ for $n \leq i$.

Equivariance of $\to_{\mathrm{d}}$: Lemma 17

Theorem 15 needs $\to_{\mathrm{d}}$ to be equivariant. $\to_{\mathrm{d}'}$ is (Theorem 10), but $\to_{\mathrm{d}}$ is built from nsub, which does not commute with permutations in the obvious way.

- Naive: permute everything. With $\pi = (0\ 1)$:

$$\pi \cdot \mathsf{nsub}\,(\mathsf{dV}\,5)\,0\,(\mathsf{dV}\,1) = \pi \cdot \mathsf{dV}\,0 = \mathsf{dV}\,1$$
$$\mathsf{nsub}\,(\pi \cdot \mathsf{dV}\,5)\,0\,(\pi \cdot \mathsf{dV}\,1) = \mathsf{nsub}\,(\mathsf{dV}\,5)\,0\,(\mathsf{dV}\,0) = \mathsf{dV}\,5$$

Lemma 17 LEMMA

$\pi \cdot (\mathsf{nsub}\,t\,i\,u) = \mathsf{nsub}\,(\pi \cdot t)\,i\,(\mathsf{inc}_i(\pi) \cdot u)$

- The index i is not permuted, and the body gets the incremented permutation: $\mathsf{inc}_0(0\ 1) = (1\ 2)$, so $\mathsf{nsub}\,(\mathsf{dV}\,5)\,0\,(\mathsf{dV}\,2) = \mathsf{dV}\,1$. Correct.
- Reason: nsub at i removes a binder, so the body lives one level deeper.
- With it, $\to_{\mathrm{d}}$ is equivariant by rule induction, and Lemma 18 follows from Theorem 15.

Theorems 19 and 20 in Lean

- Thm. 19: $t \to_{\mathrm{d}'} u \iff t \to_{\mathrm{d}} u$ LEAN
- Thm. 20 = Thm. 12 + Thm. 19 LEAN

As a Mathlib relation isomorphism LEAN:

```
noncomputable def dbRelIso :
  BetaAlpha (Var := Atom) ≃ᵣ Beta :=
  ⟨dbEquiv, betaAlpha_iff_beta.symm⟩
```

- $r \simeq_r s$: an equivalence e with $s\,(e\,a)\,(e\,b) \iff r\,a\,b$, exactly the paper's notion of isomorphic systems.
- No `sorry`; Thm. 20 depends only on `propext`, `Classical.choice`, `Quot.sound`.
- f^{-1} comes from surjectivity by choice, so it is not computable.

Payoff: Church–Rosser for free

- #475 proves Church–Rosser for $\to_{\mathrm{d}}$ [LEAN].
- An isomorphism of reduction systems transports everything stated in terms of the relation: $\twoheadrightarrow$, confluence, normal forms.

Corollary

$\to_\beta$ on Λ_α is confluent [LEAN]: if $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$, there is P with $N_1 \twoheadrightarrow_\beta P$ and $N_2 \twoheadrightarrow_\beta P$.

Corollary

β-normal forms on Λ_α are unique [LEAN].

- No further work on the named side; not in [7].

Departures from the papers

- Crole's PI is stated with renaming instead of a swap; equal under its side condition.
- $=_\alpha$ is reflexive, symmetric, transitive by direct proofs (from #458), not via Theorem 4.4.
- Λ_α substitution is lifted from raw terms, and so is f; no Pitts recursion.
- Atoms: natural numbers in a wrapper, where the paper has strings with a bijection to $\mathbb{N}$.
- Permutations are finitely supported bijections, not lists of pairs.
- lift, nsub, sub are #475's operations; the paper's equations are lemmas.
- No general notion of equivariant relation: Thm. 15 takes equivariance as a hypothesis.

Status and future work

- Crole: follow-up PRs after #741 (Thms. 4.2, 4.4–4.6, Prop. 4.3).
- Norrish and Vestergaard: cleanup under way; upstream after #475 and #741.
- The η version of Thm. 20 is also proved LEMN.
- Next:
  - more representations (locally nameless, nominal), connected by explicit isomorphisms;
  - Crole-style equivalences elsewhere, e.g. SK combinators;
  - a *formalized Barendregt* [1].

References I

[1] Hendrik Pieter Barendregt.
The lambda calculus - its syntax and semantics, volume 103 of *Studies in logic and the foundations of mathematics*.
North-Holland, 1985.

[2] Roy L. Crole.
Alpha equivalence equalities.
Theoretical Computer Science, 433:1–19, 2012.

[3] CSLib: The Lean computer science library.
`https://github.com/leanprover/cslib`.
GitHub repository.

[4] Chris Anto Fröschl.
Alpha equivalence equalities.
CSLib pull request #741, `https://github.com/leanprover/cslib/pull/741`, 2026.
Under review.

[5] Murdoch J. Gabbay.
Foundations of nominal techniques: logic and semantics of variables in abstract syntax.
Bulletin of Symbolic Logic, 17(2):161–229, 2011.

References II

[6] Tobias Nipkow.
More Church-Rosser proofs (in Isabelle/HOL).
Journal of Automated Reasoning, 26:51–66, 2001.

[7] Michael Norrish and René Vestergaard.
Proof pearl: De bruijn terms really do work.
In Klaus Schneider and Jens Brandt, editors, *Theorem Proving in Higher Order Logics*, pages 207–222, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.

[8] Zayn Wang and Maksym Radziwill.
de Bruijn syntax for untyped lambda calculus and a proof of Church-Rosser with parallel reduction.
CSLib pull request #475, `https://github.com/leanprover/cslib/pull/475`, 2026.
Open.

[9] Haoxuan Yin and Fabrizio Montesi.
Basics of named representation (alpha equivalence and capture-avoiding substitution).
CSLib pull request #458, `https://github.com/leanprover/cslib/pull/458`, 2026.
Merged.